Course : AAD – ALGORITHM ANALYSIS AND DESIGN

Chapter 6

Design and analysis of Dynamic Algorithms

Dr Rahaf Ghazal

2025-2026

Introduction

Dynamic Programming is an algorithmic technique with the following properties:

- It is mainly an optimization over plain recursion. Wherever we see a recursive solution that has repeated calls for the same inputs **(overlapping subproblems)**, we can optimize it using Dynamic Programming.
- The idea is to simply store the results of subproblems so that we do not have to re-compute them when needed later. This simple optimization typically reduces time complexities from exponential to polynomial.

When to Use Dynamic Programming (DP)?

Dynamic programming is used for solving problems that consists of the following characteristics:

1. Optimal Substructure:

means that we use the optimal results of subproblems to achieve the optimal result of the bigger problem.

2. Overlapping Subproblems:

The same subproblems are solved repeatedly in different parts of the problem.

Approaches of Dynamic Programming (DP)

Dynamic programming can be achieved using two approaches:

1. Top-Down Approach (Memoization):

- We keep the solution recursive and add a memoization table to avoid repeated calls of same subproblems.
- Before making any recursive call, we first check if the memoization table already has solution for it.

2. Bottom-Up Approach (Tabulation):

- We write an iterative solution (avoid recursion overhead) and build the solution in bottom-up manner.
- We use a dp table where we first fill the solution for base cases and then fill the remaining entries of the table using ***recursive formula***.

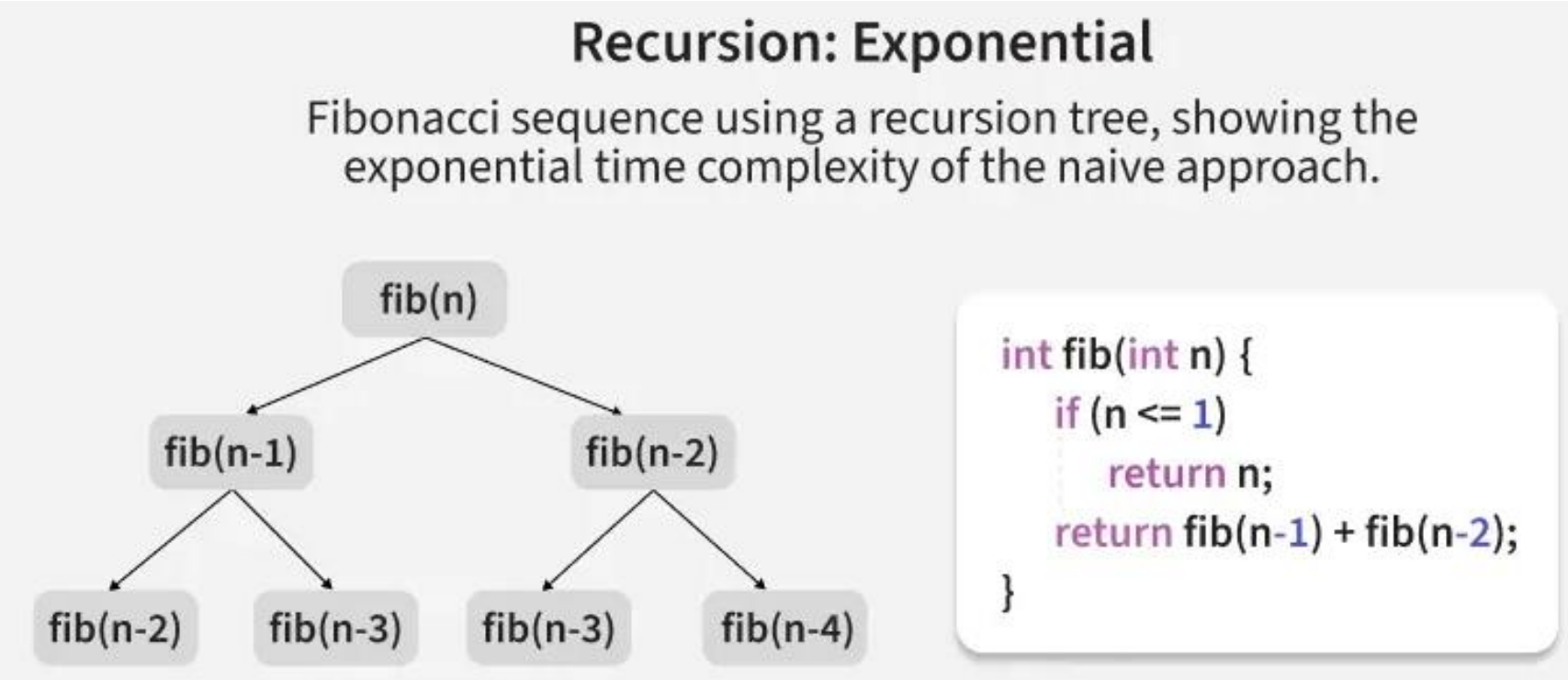
Common Algorithms that Use DP

- Fibonacci Sequence
- Knapsack Problem
- Longest Common Subsequence (LCS)
- Traveling Salesman Problem

Example 1: Fibonacci Sequence

Consider the problem of finding the Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, …

Using traditional recursion Approach(Brute Force Search):



The time complexity:

is upper bounded by $O(2^n)$ as we make two recursive calls in every function.

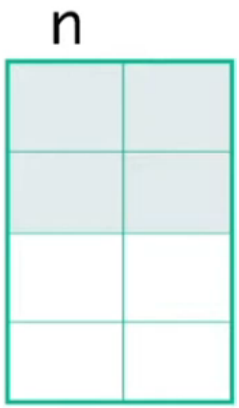
Using Top-Down Approach (Memoization):

Pseudo Code – Fibonacci (DP - Memoization)

```
 1:Input: N
 2:Output: Fibonacci number at N
 3:
 4: initiate lookup[N+1] array with -1's
 5: function Fib(n)
 6:   if lookup[n] != -1
 7:      return lookup[n]
 8:   else if n = 0 or n = 1
 9:      lookup[n] = n        OR return n
10:   else
11:      lookup[n] = Fib(n - 1) + Fib(n - 2)
12:   return lookup[n]
```

Time Complexity: O(n), each Fibonacci number is calculated only one times from 1 to n

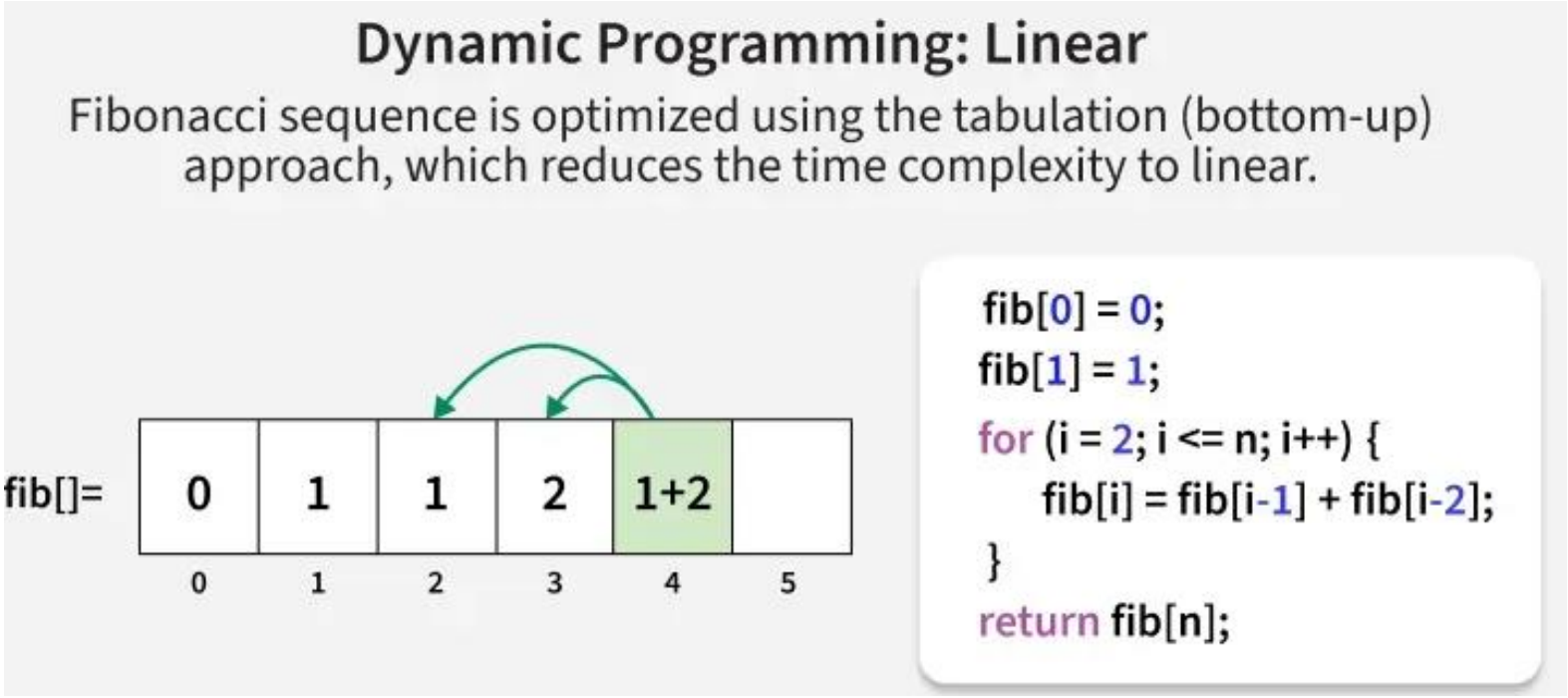




Memoized Recursive Fibonacci calls

Using Bottom-Up Approach (Tabulation):

we use an array of size (n + 1), often called **dp[]**, to store Fibonacci numbers. The array is initialized with base values at the appropriate indices, such as **dp[0] = 0 and dp[1] = 1.** Then, we iteratively calculate Fibonacci values from dp[2] to dp[n] by using the relation **dp[i] = dp[i-1] + dp[i-2]**. Finally, the value at **dp[n]** gives the Fibonacci number for the input n.



Time Complexity:

O(n), the loop runs from 2 to n, performing a constant amount of work per iteration.

Example 2: 0/1Knapsack Problem

بفرض وجود n سلعة، لكل سلعة وزن وربح مرتبط بها، وحقيبة سعتها (W) أي أن الحقيبة تتسع لوزن Wكحد أقصى، فإن المطلوب هو وضع السلع في الحقيبة بحيث يكون مجموع الأرباح المرتبطة بها هو الحد الأقصى الممكن.(أي ماهو الربح الأعظمي الذي يمكن الحصول عليه وفقا لشروط المسألة)

Index	Weight	Value
i	wgt[i-1]	val[i-1]
1	10	50
2	20	120
3	30	150
4	40	210
5	50	240



Maximum value: **270**

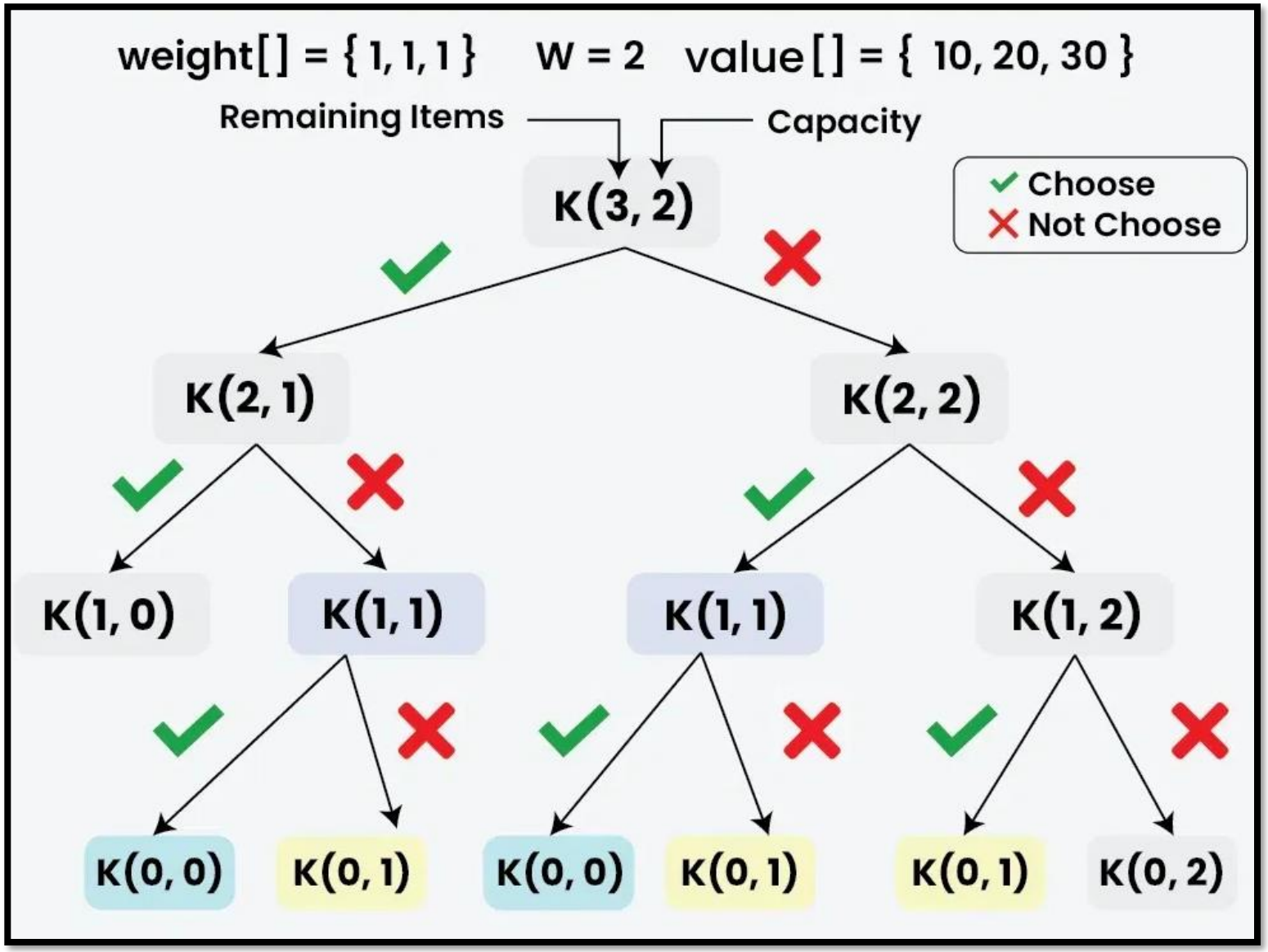
Optimal solution: Put into the backpack,
occupying **50** backpack capacity in total

Knapsack algorithm-Using recursion Approach(Brute Force Search):

Pseudocode:

```
K ( i , W ) // where i is remaining items' number (so, it begin from n value)
{  // Base case: if all items have been selected or knapsack has no capacity
        if ( i = 0 or W = 0)   return 0
   // Pick nth item if it does not exceed the capacity of knapsack
        if (wt[i] <= W)
        pick = val[i] + K ( i-1, W - wt[i])
   // Don't pick the nth item
        notPick = K ( i-1 , W)
   return max(pick, notPick);
}
```

كل عنصر من العناصر المراد وضعها في الحقيبة يملك خيارين(قرارين)، إما أن يوضع في الحقيبة أو لا. بالتالي شجرة الحل ستكون شجرة ثنائية.



Time Complexity: $O(2^n)$, since each item generates two search branches of not selecting and selecting.

Observing the recursion tree, it is easy to see overlapping subproblems

Knapsack algorithm-Using Top-Down DP (Memoization) Approach:

To ensure that overlapping subproblems are only computed once, we use a **memo** list to record the solutions to subproblems.

Pseudocode:

```
dp( i , W )
{  // Base case: if all items have been selected or knapsack has no capacity
        if ( i = 0 or W = 0)       return 0
   // Check if we have previously calculated the same subproblem
       if(memo[i][W] != -1)       return memo[i][W];
   // Pick ith item if it does not exceed the capacity of knapsack
      if (wt[i] <= W)          pick = val[i] + dp( i-1 ,W - wt[n - 1])
   // Don't pick the ith item
       notPick = dp( i-1 ,W)
   // Store the result in memo[i][W] and return it
       return memo[i][W] = max(pick, notPick);}
```

Time Complexity:

depends on the number of subproblems, which is $O(n \times W)$

Knapsack algorithm-Using Bottom-Up DP (Tabulation)Approach:

Is essentially the process of filling the table during state transitions

Pseudocode:

```
Knapsack ( W , n)
{  // Build table dp[][] in bottom-up manner
   for (int i = 0; i <= n; i++) {
      for (int j = W; j >= 0; j--) {
         // If there is no item or the knapsack's capacity is 0
         if (i == 0 || j == 0)       dp[i][j] = 0
         else {
   // Pick ith item if it does not exceed the capacity of knapsack
            if(wt[i ] <= j)        pick = val[i] + dp[i - 1][j - wt[i ]]
            // Don't pick the ith item
           notPick = dp[i - 1][j]
           dp[i][j] = max(pick, notPick)   }    }    }
   return dp[n][W]   }
```

Time Complexity: Is determined by the size of the array dp, which is **O(n x W)**

Homework

حاول أن تكتب خوارزمية ديناميكة لحساب عاملي عدد صحيح موجب.